

feedzai

logger module-attribute

```
logger = getLogger(__name__)
```

Feedzai

```
Feedzai(pulse: Server | None = None)
```

Bases: object

Entry point to ds_api functionalities.

Parameters:

Name	Type	Description	Default
<code>pulse</code>	<code>Server</code>	Server used to communicate with Pulse. If not specified default ds_api.Server will be used.	None
<code>_remote_session</code>		Most recent SparkSession created (used as local cache of the SparkSession object)	required
<code>_remote_session_params</code>		Parameters of the most recent SparkSession created	required

pulse instance-attribute

```
pulse = connect()
```

app

```
app(**kwargs)
```

Wrapper around ds_api.App constructor to use Pulse Server configured in this instance.

Other Parameters:

Name	Type	Description
<code>address</code>	<code>str</code>	The hostname.
<code>port</code>	<code>int</code>	The RMI port.
<code>timeout</code>	<code>int</code>	Maximum time to wait until timeout.
<code>user_id</code>	<code>str</code>	The user id used during RMI connections.

Returns:

Type	Description
<code>App</code>	New App instance.

`get_spark_local_session` staticmethod [¶](#)

```
get_spark_local_session(spark_params: Optional[Mapping[str, str]] = None, spark_app_name: Optional[str] = None, log_level: SparkLogLevel = ERROR)
```

Get SparkSession configured in local mode. It will use all available CPUs and each executor has 1G RAM and 1 vcore.

Note that Spark maintains a single activate session, thus unless it was crashed, you would always get the same one, but configured in local mode. Even if you stop SparkSession or SparkContext, in pySpark, it will remember previous configurations.

This means the following:

1. Previously spark had properties A, B, and C defined
2. You call this function and you will get a SparkSession with A, B, and C defined as well as the local mode

Parameters:

Name	Type	Description	Default
<code>spark_params</code> ¶	<code>dict</code>	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be found at https://spark.apache.org/docs/2.3.0/configuration.html .	<code>None</code>
<code>spark_app_name</code> ¶	<code>str</code>	The name for the spark application. If 'None' then one is automatically assigned.	<code>None</code>
<code>log_level</code> ¶	<code>SparkLogLevel</code>	Log Level of the spark session, defaults to <code>SparkLogLevel.ERROR</code> .	<code>ERROR</code>

Returns:

Type	Description
<code>SparkSession</code>	A Spark Session in local mode, None if spark_params is not a dictionary.

`get_spark_session` [¶](#)

```
get_spark_session(spark_params: Dict[str, str] | None = None, exclude_spark_params: List[str] | None = None, app_name: str | None = None, spark_app_name: str | None = None, job_profile_name: str | None = None, group_name: str | None = None, clean_spark_properties: bool = True, log_level: SparkLogLevel = ERROR, session_type: SparkSessionType = CLASSIC, spark_remote: str | None = None)
```

Get SparkSession with default Pulse's configuration.

Note that Spark maintains a single activate session, thus unless it was crashed, you would always get the same one, but with last Pulse's configuration (and possible old configurations). Even if you stop SparkSession or SparkContext, in pySpark, it will remember previous configurations.

This means the following:

1. Pulse has properties A, B, and C defined

2. You call this function and you will get a SparkSession with A, B, and C properties
3. You remove property A and add property D
4. You call this function and you will get a SparkSession with **A**, B, C, and D

Parameters:

Name	Type	Description	Default
<code>spark_params</code> ↗	dict	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be find in https://spark.apache.org/docs/latest/configuration.html	None
<code>exclude_spark_params</code> ↗	list	List with Spark properties to be removed. Full list of Spark properties can be find in https://spark.apache.org/docs/latest/configuration.html	None
<code>app_name</code> ↗	str	An optional name of a pulse application. This parameter should be used when we want to have access to application specific properties. This allows to use the same properties being used by Spark jobs executed in Pulse.	None
<code>spark_app_name</code> ↗	str	The name for the spark application. If 'None' then one is automatically assigned.	None
<code>job_profile_name</code> ↗	str	The name of the job profile to reuse the Spark properties defined there. If 'None' then it will retrieve the properties associated to default job profile for jobs of type 'EXTERNAL_DS_JOB'.	None
<code>group_name</code> ↗	str	The name of the group model used to retrieve the configurations defined in its properties.	None
<code>clean_spark_properties</code> ↗	bool	True if it should clean the Spark properties specific to Pulse use case, false otherwise.	True
<code>log_level</code> ↗	SparkLogLevel	Log Level of the spark session, defaults to SparkLogLevel.ERROR.	ERROR
<code>session_type</code> ↗	SparkSessionType	The type of Spark Session to be created, defaults to <code>SparkSessionType.CLASSIC</code>	CLASSIC
<code>spark_remote</code> ↗	str	The URL of the remote Spark Connect server	None

Returns:

Type	Description
<code>_TSparkSession</code>	A SparkSession with Pulse's configuration, None if spark_params is not a dictionary

pandas_to_pulse staticmethod [↗](#)

```
pandas_to_pulse(dataframe: DataFrame, app: App, name: str, path: str, file_format: str = 'csv', schema: Optional[Schema] = None, **kwargs: Union[bool, str, int, Dict[str, Any], Set[str]])
```

Converts a given pandas DataFrame to Pulse datasource using SparkSession#createDataFrame method of pySpark dataframe.

Parameters:

Name	Type	Description	Default
<code>dataframe</code> ↗	DataFrame	DataFrame which will be used to create a Pulse datasource.	<i>required</i>
<code>app</code> ↗	App	The app in which the datasource will be created.	<i>required</i>
<code>name</code> ↗	str	The name of the new datasource.	<i>required</i>
<code>path</code> ↗	str	The path of the new datasource.	<i>required</i>
<code>file_format</code> ↗	str	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	'csv'
<code>schema</code> ↗	Schema	The pulse schema used to create the datasource. If not specified, the schema will be inferred.	None

Other Parameters:

Name	Type	Description
<code>has_header</code>	bool	Flags whether the dataset is to be read with header. Defaults to True.
<code>sep</code>	str	Delimiter to use. Should contain exactly one character. Defaults to ','.
<code>missing_values</code>	set of str	Set of strings with values for empty value. Defaults to { "", "NA", "NULL" }.
<code>quote</code>	str	The character used to denote the start and end of a quoted item. Defaults to "".
<code>line_separator</code>	str	The string used mark the end of line. Defaults to "\n".
<code>encoding</code>	str	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'.
<code>cat_threshold</code>	int	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
<code>job_properties</code>	Dict	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse Properties Profile.
<code>n_records</code>	int	

Name	Type	Description
		The number of records that will be read per partition when performing datasource schema inference. Defaults to 15000. Will only be used if no Schema is specified.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Default to None.
<code>merge_schemas</code>	<code>bool</code>	Specify if schema merging should be allowed. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
<code>DataSource</code>	The new <code>ds_api.DataSource</code> instance.

`pulse_to_pandas` staticmethod [¶](#)

```
pulse_to_pandas(app: App, name: str, spark_properties: dict | None = None)
```

Converts a given datasource (by app and name) to a pandas DataFrame using toPandas method of pySpark dataframe.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Application name, where the datasource is persisted.	<i>required</i>
<code>name</code> ¶	<code>str</code>	The DataSource name.	<i>required</i>
<code>spark_properties</code> ¶	<code>Dict</code>	Dictionary with Spark parameters to be overridden or added. Full list of parameters can be found in https://spark.apache.org/docs/2.3.0/configuration.html .	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	A pandas DataFrame correspondent to given pulse-based DataSource.

`pulse_to_pyspark` staticmethod [¶](#)

```
pulse_to_pyspark(spark_session, app, name)
```

Uses `ds_api.SparkDataSourceAdapter` to convert a given data source (by app and name), to a pyspark DataFrame.

Parameters:

Name	Type	Description	Default
<code>spark_session</code> ¶	<code>pyspark.sql.SparkSession</code>		<i>required</i>

Name	Type	Description	Default
<code>app</code> ⌕	<code>App</code>	Application name, where the Data Source is persisted.	<i>required</i>
<code>name</code> ⌕	<code>str</code>	The Data Source name as it appears in the UI.	<i>required</i>

Returns:

Type	Description
<code>DataFrame</code>	A pyspark DataFrame correspondent to given pulse-based Data Source.

`pyspark_to_pulse` [staticmethod](#) [⌕](#)

```
pyspark_to_pulse(spark_session: _TSparkSession, dataframe: _TDataFrame, app: App, name: str,
path: str, file_format: str = 'csv', schema: Optional[Schema] = None, **kwargs: Union[bool, str,
int, Dict[str, Any], Set[str]])
```

Uses `ds_api.SparkDataSourceAdapter` to convert a given pySpark dataframe to a Pulse datasource.

The dataframe will be written in the path specified before being added to Pulse. `DateType` and `TimestampType` fields are written as unix time (in seconds). The schema will be automatically inferred using `AbstractDataSource#inferSchema` approach if the Schema is not specified.

All pySpark types but `DecimalType` and `Mllib` types are supported.

Parameters:

Name	Type	Description	Default
<code>spark_session</code> ⌕	<code>pyspark.sql.SparkSession</code>	The Spark session.	<i>required</i>
<code>dataframe</code> ⌕	<code>DataFrame</code>	DataFrame which will be used to create a Pulse datasource.	<i>required</i>
<code>app</code> ⌕	<code>App</code>	The app in which the datasource will be created.	<i>required</i>
<code>name</code> ⌕	<code>str</code>	The name of the new datasource.	<i>required</i>
<code>path</code> ⌕	<code>str</code>	The path of the new datasource.	<i>required</i>
<code>file_format</code> ⌕	<code>str</code>	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	<code>'csv'</code>
<code>schema</code> ⌕	<code>Schema</code>	The pulse schema used to create the datasource. If not specified, the schema will be inferred.	<code>None</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Defaults to ','.
<code>missing_values</code>	<code>set of str</code>	Set of strings with values for empty value. Defaults to { "", "NA", "NULL" }.
<code>quote</code>	<code>str</code>	The character used to denote the start and end of a quoted item. Defaults to "".
<code>line_separator</code>	<code>str</code>	The string used mark the end of line. Defaults to "\n".
<code>encoding</code>	<code>str</code>	Encoding to use for UTF when reading/writing. Defaults to 'UTF-8'.
<code>cat_threshold</code>	<code>int</code>	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15. Will only be used if no Schema is specified.
<code>job_properties</code>	<code>Dict</code>	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse Properties Profile. Deprecated in version 25.0.41 and will be removed.
<code>n_records</code>	<code>int</code>	The number of records that will be read per partition when performing datasource schema inference. Defaults to 15000. Will only be used if no Schema is specified.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Default to None.
<code>merge_schemas</code>	<code>bool</code>	Specify if schema merging should be allowed. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
<code>DataSource</code>	New DataSource instance.

SparkLogLevel

Bases: `Enum`

ALL `class-attribute` `instance-attribute` 

ALL = 'ALL'

DEBUG `class-attribute` `instance-attribute` 

DEBUG = 'DEBUG'

ERROR `class-attribute` `instance-attribute` 

ERROR = 'ERROR'

FATAL class-attribute instance-attribute [Â¶](#)

```
FATAL = 'FATAL'
```

INFO class-attribute instance-attribute [Â¶](#)

```
INFO = 'INFO'
```

OFF class-attribute instance-attribute [Â¶](#)

```
OFF = 'OFF'
```

TRACE class-attribute instance-attribute [Â¶](#)

```
TRACE = 'TRACE'
```

WARN class-attribute instance-attribute [Â¶](#)

```
WARN = 'WARN'
```

SparkSessionType [Â¶](#)

Bases: Enum

CLASSIC class-attribute instance-attribute [Â¶](#)

```
CLASSIC = 'CLASSIC'
```

CONNECT class-attribute instance-attribute [Â¶](#)

```
CONNECT = 'CONNECT'
```